Unbounded Viewstamped Replication Revisited for Diskless Strong Consistency

Simon Massey

***Abstract*—Small replicated metadata should not require a disk flush on the critical path. Unbounded Viewstamped Replication Revisited is a protocol design for embedding strong consistency with volatile normal-path replication and crash-stop reincarnation. A cleanly stopped process restarts with its identity and flushed state. A crashed process permanently retires its old identity and rejoins under a fresh identity, obtaining state and voting authority through reconfiguration. Its organising mechanism is leader overlap: where the leader is the sole intersection of the active decision quorum and the next prepare quorum, it withholds its own promise so that preparation can proceed while the active quorum remains usable. For a five-voter, unit-weight configuration, replacement takes two committed configuration batches: first removing the old vote, then enabling the new vote. The Lean development proves per-slot agreement across this schedule from the protocol's history invariants, verifies the replacement weights, and establishes both exclusion of the evicted voter and nonreuse of its identity.**

***Index Terms*—Crash-stop, leader overlap, reconfiguration, strong consistency, Viewstamped Replication.**

I. Introduction

Oki and Liskov introduced Viewstamped Replication in 1988 [1]. Liskov and Cowling's 2012 Viewstamped Replication Revisited (VRR) eliminated disk writes during normal processing and view changes [2]. Replication therefore retained the state needed for agreement without adding a local disk flush to each update. The remaining difficulty was how to bring a replica back after loss of that state.

Michael *et al.* exposed a safety failure in VRR's recovery mechanism in 2016 [3, Sec. 4.2 and Appendix A.1]. A replica can send a view-change commitment, crash, and return in an older view having forgotten the commitment. It can then help commit an operation that a later leader loses when delayed view-change messages arrive. Their counterexample breaks linearizability, including when channels preserve message order. The authors provide a virtual stable storage construction to support diskless crash recovery. Their result makes the problem precise: restoring a replica's data must also preserve the commitments made by its identity.

Unbounded Viewstamped Replication Revisited (uVRR) repairs this failure by making a crash final for the protocol identity. The crashed identity never resumes execution. The process that replaces it joins under a fresh identity, receives state from the cluster, and acquires voting authority through committed membership changes. Messages sent before the crash remain messages of the old identity; the new process cannot add fresh votes to that identity's history. A cleanly stopped process can instead restart under its existing identity because it has preserved its protocol state. Startup fencing and clean shutdown use durable writes, keeping those writes outside normal replication. Loss of volatile state therefore leads to replacement of a member: crash recovery becomes crash-stop reincarnation.

Turner supplies the construction that connects this lifecycle to continued replication [4]. Where the leader is the sole intersection of an active decision quorum and the next prepare quorum, it can withhold its own promise while collecting the others. The active quorum remains usable during that preparation; the leader completes the handover with its own local promise. We use this leader overlap to organise reincarnation. For five unit-weight voters, the replacement schedule has two committed configuration batches: remove the old vote and admit the replacement without a vote, then enable the replacement's vote. The schedule includes a leader-overlap pivot at the second boundary. Turner's agreement argument supplies the basis for proving that view changes across the schedule preserve decisions.

Together, these ideas retain VRR's diskless normal path while giving failed members a route back through fresh identities. The intended setting is agreement over small amounts of metadata embedded in a distributed application [5]. Such a service can coordinate membership or advisory-lock metadata without putting synchronous persistence on every update's critical path. Its normal replication need not wait for a disk flush or batch updates to amortise one.

Section II develops the lifecycle and leader-overlap construction. Section III gives the five-voter schedule and its Lean proof of per-slot agreement from the protocol's history invariants, together with the replacement weights, exclusion of the evicted voter and identity nonreuse. Section IV develops the metadata applications, and the appendix sets out the mathematical hypotheses and verification evidence. The name "unbounded" follows Turner's treatment of pipelining through reconfiguration: agreement does not depend on a fixed bound on outstanding slots.

II. Method

A. Processes, state and identity

The agreement model is asynchronous and non-Byzantine. Messages may be delayed, and an old message retains the identity that issued it. Each incarnation has a logical identity distinct from its physical host. The lifecycle must ensure that an identity never produces new protocol actions after a replacement incarnation takes over.

Author draft. Correspondence: simon.massey@stenographer.cloud.

A clean stop first quiesces protocol processing, then flushes the state required for restart, and finally records a durable stopped marker. On startup, a valid stopped marker permits the process to reload that state and enter *Restarting* with the same identity. Before sending protocol messages it durably records that it is running again. This boot fence prevents a later crash from being mistaken for another clean restart. A restarting process retains its promises and accepted state, so ordinary timeout processing is permitted by the design.

Without a valid clean-stop marker, startup enters *Joining* under a fresh identity. The old identity is crash-stopped. Cached state may reduce transfer work, but it provides no voting authority. The joiner asks the leader to evict the old incarnation and admit the new one; the leader can send missing state while coordinating the change. A joiner does not vote or initiate a view change. Its authority comes from the committed configuration, never from the fact that it shares a host with a former voter.

The implementation design uses replicated superblock markers and majority interpretation of valid marker copies. This paper abstracts that mechanism as correct clean-stop classification and durable identity fencing. The Lean result concerns the resulting lifecycle; it does not model the storage protocol itself.

B. Quorums across eras

An era determines the quorum families for a suffix of log slots. Write $\mathcal{Q}_e^{\mathrm{I}}$ for its phase-one family and $\mathcal{Q}_e^{\mathrm{II}}$ for its phase-two family. The history model requires the following within-era and adjacent-era intersections:

$$q \cap r \neq \varnothing \qquad (q \in \mathcal{Q}_e^{\mathrm{II}},\ r \in \mathcal{Q}_e^{\mathrm{I}}), \tag{1}$$

$$q \cap r \neq \varnothing \qquad (q \in \mathcal{Q}_e^{\mathrm{I}},\ r \in \mathcal{Q}_{e+1}^{\mathrm{II}}). \tag{2}$$

The direction in (2) matters: the earlier phase-one quorum must meet the later decision quorum. These conditions are part of Turner's era-based agreement argument [4].

Our concrete schedule uses the same strict weighted-majority family for both phases. If $w_e(n)$ is a nonnegative integer weight on a finite support, a quorum q satisfies

$$2\sum_{n\in q} w_e(n) > \sum_{n} w_e(n). \tag{3}$$

The weighted proof establishes the requisite intersections when successive configurations differ by at most one unit of total absolute weight. Membership records for zero-weight nodes may change in the same batch without altering (3). A crash itself does not change any weight; only committed configuration commands do.

C. The leader as the overlap

Suppose a leader ℓ uses ballot b with an active decision quorum q. To prepare ballot $b' > b$ for the next era, it chooses a prospective phase-one quorum q' with

$$q \in \mathcal{Q}_e^{\mathrm{II}}, \qquad q' \in \mathcal{Q}_{e+1}^{\mathrm{I}}, \qquad q \cap q' = \{\ell\}. \tag{4}$$

This is Turner's casting-vote condition [4]. The leader first sends prepare requests only to $q' \setminus \{\ell\}$. Their promises do not disable an acceptor in q. The leader withholds its own promise and may continue proposing at b, provided no other event invalidates the old ballot's guards.

After receiving those promises, the leader chooses the suffix boundary k and makes its own promise locally. Phase one is then complete without another network exchange for the leader's response. Proposals at b' must still obey the normal rule for preserving values reported by phase one. Figure 1 shows the message order.

Decision quorums follow the era of the *slot*, not simply the era associated with a ballot. Consequently, continuing to decide later-era slots at b also requires a quorum valid for those slots. In the concrete pivot below, the active quorum is a majority in both adjoining eras.

TABLE I
VOTING WEIGHTS FOR THE TWO REPLACEMENT BATCHES. A ZERO RECORDS VOTING WEIGHT, NOT WHETHER A ZERO-WEIGHT MEMBERSHIP RECORD IS PRESENT.

Era	n_0	n_1	n_2	n_3	n_4	n_5
E_0	1	1	1	1	1	0
E_1	0	1	1	1	1	0
E_2	0	1	1	1	1	1

D. Replacing one of five voters

Let n_0 be the failed incarnation, $n_1, \ldots, n_4$ the other original voters, and n_5 its fresh replacement. Table I gives the schedule. In the first committed batch, the leader removes n_0's voting weight and joins n_5 at weight zero. Once the replacement is ready, the second batch gives it unit weight and removes the old zero-weight membership record. Each boundary changes the weight vector by one unit.

At the second boundary, $q = \{n_1, n_2, n_3\}$ is a majority in both E_1 and E_2, and $q' = \{n_3, n_4, n_5\}$ is a majority in E_2. Their only common member is n_3, which can serve as the leader in (4). This is the explicit five-node pivot proved by the Lean development.

The first boundary has a different geometry. Any surviving decision quorum in E_0 needs at least three of the four survivors, and every majority in E_1 also needs three of them. Two such sets share at least two members. Hence there is no singleton leader overlap for this boundary in the unit-weight example. The two batches are safe under the agreement assumptions. Preparation, state transfer, announcements and retransmission remain part of the protocol.

III. Results

A. The agreement result

The final Lean theorem composes an era-based agreement theorem with the concrete replacement weights and an abstract identity lifecycle. Its contract assumes a well-founded total ballot order, a ballot-to-era map monotone in ballot order, the required ballot/slot era bound, and the history invariants corresponding to Turner's P2–P7 [4]. Appendix A states the assumptions and reproduction procedure. Those invariants constrain promises, reports of prior acceptances, phase-one

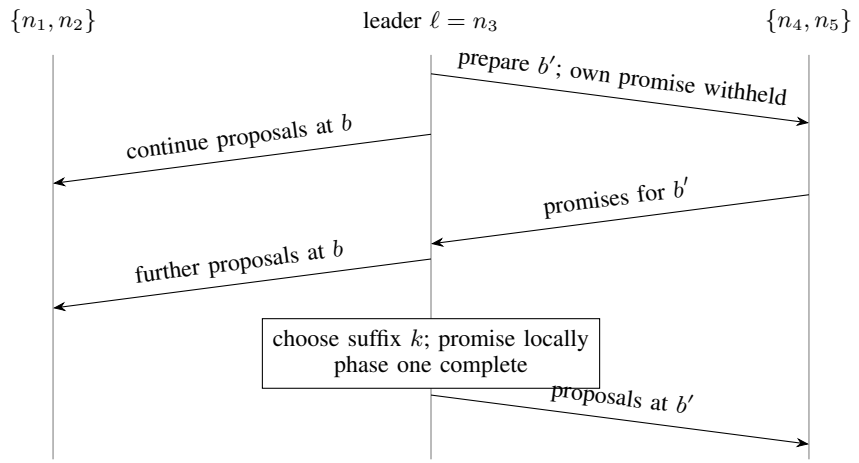

Fig. 1. Leader overlap at the $E_1 \to E_2$ pivot: $q = \{n_1, n_2, n_3\}$ and $q' = \{n_3, n_4, n_5\}$. Original schematic of Turner's casting-vote construction [4], specialised to the replacement schedule. Time runs downwards without a scale. Group arrows represent messages to each member; acceptance replies, state transfer and unrelated traffic are omitted.

justification and value selection, acceptance of proposals, and quorum evidence for a chosen value. The theorem derives quorum intersection from the weight schedule.

Theorem 1 (Five-voter replacement)**.** *For a history whose phase-one and phase-two quorum families follow Table I, assuming the ballot-order, era and history conditions above, any two values chosen for the same slot are equal. In E_1, both the failed identity and its replacement have zero weight; in E_2, the failed identity has zero weight and its replacement has unit weight. From completion of the crossing era onwards, the old identity has no voting weight in any continuation of the forced configuration run. Separately, along an identity run satisfying the formal lifecycle transitions, an identity that has been superseded by a bump is never again the current identity.*

Proof. The strict-majority construction and one-unit boundaries establish the within-era and adjacent-era quorum intersections (1)–(2), together with quorum nonemptiness. Instantiating the era-based agreement theorem with those families establishes equality of chosen values under the supplied history invariants. Evaluation of the concrete configurations establishes the stated weights. The eviction invariant preserves zero weight for the old identity along the forced run. The identity lifecycle transitions preserve monotonic identity growth, excluding a superseded identity from becoming current again. □

The leader-overlap lemmas establish that preparing members outside the active quorum preserves that quorum's guards, and that adding the leader's promise completes the prospective phase-one quorum.

B. Evidence and reproduction

The Lean 4 project is archived at repository commit `352b0bf` [11], extended here by adding the existing eviction invariant to the final conjunction. It uses Lean 4 version 4.33.1 [12] without Mathlib. The source directory is `formal/uvrr-lean/UVRR/`. The weighted intersection, era agreement, casting-vote and five-voter modules supply the components of Theorem 1. Its final statement is `ReincarnationSafety.five_safe`, in `ReincarnationSafety.lean`, importing `ReincarnationAgreement.lean`.

From `formal/uvrr-lean`, run `lake build` and `python3 check_axioms.py`. The checked baseline's axiom audit covers 355 declarations and permits only Lean's standard logical axioms; it rejects proof holes and custom axioms. The recorded rung-26 evidence also exercises a negative control that tries to give the replacement voting weight in E_1. Its failure checks that particular erroneous claim. The repository contains the proof-generation transcript; generated proof terms are checked by Lean, and the generator is not part of the trusted proof checker.

The theorem covers the displayed replacement schedule with its final configuration retained thereafter. The general weighted-intersection lemmas have broader scope than this instantiation.

IV. Discussion

A. Applications

The intended deployment is an embedded authority for small metadata, with ordinary replication performed in memory.

a) Short-lived advisory locks and leader leases: A replicated state machine can order acquisition, renewal and expiry decisions over a small record containing an owner, generation and expiry policy. An expiry command for an old generation must not revoke a newer grant. If a client uses ownership to control an external resource, that resource may also need to reject stale generations. A real-time leader lease additionally needs explicit clock and timing assumptions.

b) A membership authority for another algorithm: An application whose bulk algorithm uses gossip or another weaker consistency model may still need one agreed version of cluster membership, ownership assignments or configuration changes. An embedded replicated state machine can supply

that authority, while the bulk algorithm disseminates or consumes its decisions.

c) The CORFU precedent: CORFU provides a concrete precedent for compact configuration metadata alongside a larger storage system [6]. Its reconfiguration is patterned after Vertical Paxos; Section 2.5 describes storage units acting as acceptors in a Paxos-like protocol. Section 2.6 estimates that adding 0.1% writes keeps projection metadata below 25 MB for a 1 TB cluster under an adversarial workload. The described implementation uses a shared network drive for agreed projections.

d) External coordination services: ZooKeeper and etcd are established ways to supply coordination separately from an application. ZooKeeper offers linearizable state changes and per-client ordering, while local reads have weaker semantics [7]. etcd documents strict serializability for its key-value operations and distinguishes default linearizable reads from optional serializable reads [8]. An application must choose the operation semantics it actually needs.

A separately operated service also brings deployment and resource costs. For scale, etcd's version 3.6 hardware guide gives a small-cluster example with dedicated two-vCPU, 8 GB AWS machines [9]. etcd can itself be embedded in a Go application [10]. The proposed combination here is small application-owned state, volatile normal-path replication, and replacement through fresh identities and overlapping quorums.

B. Scope

a) Implementation correspondence: The final theorem assumes protocol-history invariants. Their composition with the era-changing operational model is left as an exercise to the reader, as specified in Appendix A. The appendix also identifies the corresponding implementation obligations: clean-stop classification, identity fencing and state transfer before voting.

b) Scope and availability: The worked example replaces one unit-weight voter among five. Other weights or simultaneous failures need an appropriate schedule and available quorums. A higher ballot, an unavailable leader or insufficient surviving voters can interrupt progress. Fresh identities permit repeated replacement in the design; the theorem covers the displayed schedule.

c) Meaning of strong consistency: The mechanised result is per-slot agreement. A complete service-level linearizability argument also needs ordered execution, appropriate read semantics and a mapping between client operations and log entries. Lease guarantees require the additional conditions stated above.

V. Related Work

Viewstamped Replication introduced the organisation of replication around views and a primary [1]; VRR revisits that organisation and separates normal operation, view change, recovery and reconfiguration [2]. Michael *et al.* identify the failure of diskless recovery to preserve view-change commitments and construct virtual stable storage [3]. uVRR takes a different route: it makes a crash final for the old identity, then brings the process back as a new member through reconfiguration. The replacement schedule connects that lifecycle to agreement across configurations.

Turner's *Unbounded Pipelining in Dynamically Reconfigurable Paxos Clusters* supplies the central quorum geometry, era-based agreement argument and casting-vote construction [4]. Withholding the leader's own promise is Turner's mechanism. The present work applies it to the five-voter replacement pivot, makes the non-singleton first boundary explicit, and proves agreement for the replacement schedule from the protocol's history invariants. The application discussion places this design alongside coordination services and CORFU, using their stated semantics and resource context.

Appendix A
Proof structure, hypotheses and executable checks

This appendix separates the mathematical argument, its mechanised components, and the obligations supplied as hypotheses. The term UPaxos below refers to Turner's paper [4]. The weighted-intersection and rescaling results follow from UPaxos Appendix A.

A. Weighted overlap, doubling and halving

Let N be a finite set of identities, with nonnegative integral weight functions w, v. Write $W = \sum_{n\in N} w(n)$, $V = \sum_{n\in N} v(n)$ and $w(q) = \sum_{n\in N\cap q} w(n)$. A strict majority has $2w(q) > W$.

Lemma 1 (Scaled overlap, UPaxos)**.** *If positive integers a, b satisfy*

$$\sum_{n\in N} |aw(n) - bv(n)| \le 1, \tag{5}$$

then every strict majority under w intersects every strict majority under v.

Proof. Positive scaling preserves strict majority. Suppose majorities q, r are disjoint. Summing the nodewise disjointness bound gives

$$2aw(q) + 2bv(r) \le aW + bV + \sum_{n\in N} |aw(n) - bv(n)|. \tag{6}$$

Since all quantities are integral, the two strict majority inequalities make the left side at least $aW + bV + 2$. Equations (5) and (6) contradict that bound. The Lean declarations are `WeightedGeneral.disjoint_bound` and `WeightedGeneral.scaled_overlap`. □

Taking $a = b = 1$ gives the one-unit change rule. Exact rescaling makes the distance zero: doubling all weights preserves the majority family, as does exact halving when all weights are even. Thus these transformations preserve the required intersections. Arbitrary rounding during halving is not covered by this argument. Lean provides `majority_scale` and `scaled_equal_overlap` in the same module. Composing an iterated replacement of a doubled-weight victim is outside the concrete result presented here.

B. Hypotheses of the agreement theorem

For each slot i and ballot b, let $e(i)$ and $e(b)$ be their eras. The contract of `ReincarnationSafety.five_safe` supplies:

1) A well-founded strict total ballot order, with $e(b)$ monotone in that order. A proposed value satisfies $e(b) \leq e(i)$.
2) Both quorum families equal the strict-majority family of the displayed E_0, E_1, E_2 sequence, with E_2 retained afterwards.
3) P2–P3: a free promise for a ballot excludes lower-ballot acceptances for that slot.
4) P4: a promise reporting a prior acceptance reports the greatest acceptance below its ballot.
5) P5: a proposal is justified by a phase-one quorum at its ballot era; if that quorum reports prior values, the proposal selects the value at a maximal reported ballot.
6) P6: every acceptance has a corresponding proposal.
7) P7: a chosen value has acceptance evidence from a phase-two quorum at the slot era, and $e(i) \leq e(b) + 1$.

These are history predicates. The fixed-configuration operational components in rungs 13–16 do not constitute a completed composition with the changing era sequence.

a) Exercise (composition): Construct the mapping from the era-changing operational execution to this history and establish the listed predicates, including preservation of the promise fence, maximal-history selection, and installation across configuration boundaries. This composition is left as an exercise to the reader. The host must additionally implement clean-stop classification, fresh identity allocation and state transfer before voting.

C. Composition steps and conclusion

First, apply Lemma 1 to each adjacent pair in Table I, and self-overlap within each era. The case split over era 0, era 1, and later eras is `ReincarnationAgreement.sequence_p1`; it derives P1 for the sequence. Second, instantiate Turner's Theorem 10 with P1 and the contract above to obtain per-slot agreement. Third, evaluate the four intermediate and final weights in `Replacement`. Fourth, apply `ReincarnationFive.five_evicted_never_voting`: after the crossing era, the old identity has zero voting authority throughout any continuation of the forced run. Finally, apply `Reincarnation.amnesia_unreachable` to exclude reuse of the pre-bump identity in the identity transition system. These are the four conjuncts now exposed by `Safe`.

Zero voting authority is a configuration property. Delayed old messages retain their old identity. The separate identity and configuration runs are abstract components; the exercise above supplies their relationship to a single executable protocol. The concrete leader-overlap witness is the E_1/E_2 pivot in Figure 1.

D. Commands and observed checks

From `formal/uvrr-lean`, the checks are:

```
lake build
python3 check_axioms.py
python3 check_mutations.py
showboat verify \
ladder/26-reincarnation-safety.md
```

The build completes 24 jobs. The axiom audit covers 355 declarations; the final theorem depends only on `propext`, `Classical.choice` and `Quot.sound`. The mutation suite checks rejection of an existential quorum checker, an unsafe decision family, an omitted next-slot guard, append after a view-change fence, and cross-view prepare receipt. It also checks detection of `sorryAx`, even when the compiler returns success. Rung 26 checks that promoting the replacement prematurely in E_1 is rejected.

The Leanstral driver has two offline regressions, run from the repository root with `python3` applied to `tests/test_leanstral_driver.py`. A mock API forces two concurrent proof loops to finish writing their candidates before either can copy its result, exposing a shared-file race deterministically. A second assertion checks that the prompt preserves project imports. Both failed on the original driver and pass with private per-invocation scratch directories and an import-preserving prompt.

TABLE II
REPORTED ISOLATED LEANSTRAL REPAIR OUTCOMES. FAILURE MEANS NO ACCEPTED REPAIR WITHIN THE STATED BUDGET.

Mutation and supplied context	Outcome
M1: delete final proof; file only	Fail
M2: M1 plus four lemma signatures	Pass, round 4
M3: wrong conjunction projection	Fail
M4: delete sequence P1 proof; signatures supplied	Pass, round 2; repeat failed
M5: false claim $c_1(5) = 1$	Fail, as required

E. Reported adversarial repair experiment

Table II records the externally supplied Fable 5.1 report, not a fresh experiment executed for this revision. The report used Leanstral 1.5, a five-round budget per loop, isolated directories, and byte-identical statement checks on passing repairs. Its initial shared-directory batch was discarded because candidates crossed between jobs. The raw repair transcripts were not supplied with the report, so the table should be read with that provenance. The contrast between M1 and M2 suggests that supplying library signatures helped composition in these runs. M5 supplies a useful false-statement control.

REFERENCES

[1] B. M. Oki and B. H. Liskov, "Viewstamped replication: A new primary copy method to support highly-available distributed systems," in *ACM PODC*, 1988, pp. 8–17.

[2] B. Liskov and J. Cowling, "Viewstamped replication revisited," MIT CSAIL, Tech. Rep. MIT-CSAIL-TR-2012-021, 2012. [Online]. Available: https://hdl.handle.net/1721.1/71763

[3] E. Michael, D. R. K. Ports, N. Kr. Sharma, and A. Szekeres, "Providing stable storage for the diskless crash-recovery failure model," University of Washington, Tech. Rep. UW-CSE-16-08-02, Aug. 25, 2016. [Online]. Available: https://syslab.cs.washington.edu/papers/diskless-tr16.pdf

[4] D. Turner, "Unbounded pipelining in dynamically reconfigurable Paxos clusters," revision 1A9DBA37, Aug. 14, 2017. [Online]. Available: https://github.com/DaveCTurner/paxos-membership. The proof artifact includes the consulted PDF and its digest.

[5] S. Massey, "Viewstamped replication revisited," Aug. 12, 2026. [Online]. Available: https://simbo1905.wordpress.com/2026/08/12/viewstamped-replication-revisited/
[6] M. Balakrishnan, D. Malkhi, J. D. Davis, V. Prabhakaran, M. Wei, and T. Wobber, "CORFU: A Distributed Shared Log," *ACM Trans. Comput. Syst.*, vol. 31, no. 4, Art. 10, Dec. 2013, doi: 10.1145/2535930.
[7] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "ZooKeeper: Wait-free coordination for Internet-scale systems," in *USENIX ATC*, 2010. [Online]. Available: https://www.usenix.org/events/usenix10/tech/full_papers/Hunt.pdf
[8] etcd authors, "etcd API guarantees," documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/learning/api_guarantees/
[9] etcd authors, "Hardware recommendations," documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/op-guide/hardware/
[10] etcd authors, "Embedding etcd in a Go application," documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/dev-guide/golang_embed_pkg/
[11] S. Massey, "uVRR proof artifact," commit 352b0bf, 2026. [Online]. Available: https://github.com/lua-lunet/uvrr-core/tree/352b0bf/formal/uvrr-lean
[12] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, 2021, pp. 625–635, doi: 10.1007/978-3-030-79876-5_37.